Convolutional Neural Networks — Part II

Dilated/Transposed Convolution, Efficient Convolution, Structured Input/Output, History of CNNs

2026-04-15

Table of contents

1 View at a glance **4**

2 Review: Basic Convolution Operations and Pooling **5**
2.1 Convolution is actually cross-correlation . . . 5
2.2 Stride: Decimation from a multirate perspective . . . 5
2.3 Padding: Preservation of boundary information . . . 6
2.4 Output Size Formula . . . 6
2.5 Counting parameters (CNN vs MLP) . . . 7
2.6 Pooling: An operation to select a number from a pool . . . 8
2.6.1 Recurrence relation . . . 9

3 Dilated (Atrous) Convolution **10**
3.1 Definition and effective kernel size . . . 10
3.2 What is the difference from Stride? . . . 10

4 Transposed Convolution (Deconvolution) **12**
4.1 Why "transposed" — Transposition of Linear Transformations . . . 12
4.2 Transposed conv with stride and its uses . . . 12
4.2.1 Fully Convolutional Network (FCN) and Semantic Partitioning . . . 13

5 Beyond Vanilla CNN 1: Efficient Convolution **14**
5.1 Separable Convolution . . . 14
5.2 Group Convolution . . . 14

6 Beyond Vanilla CNN 2: Structured Input/Output and Input Forms **16**
6.1 Structured outputs . . . 16
6.2 Various input types (Input data types) . . . 16

7 Beyond Vanilla CNN 3: Random / Unsupervised Filter **18**

8 The neuroscience basis of CNN **19**

9 Why CNN Rises: ImageNet and a Brief History **20**
9.1 ImageNet — Large image dataset . . . 20
9.2 LeNet → AlexNet → Deeper . . . 20
9.3 What's next? Transformer / ViT / gMLP . . . 21

10 Key Summary **22**

11 Appendix: Check directly with NumPy **24**
11.1 Output size·Parameters·Area . . . 24
11.2 2D cross-correlation using im2col method (including dilated) . . . 24

11.3 Transposed convolution = Transpose of the convolution matrix 25
11.4 Separable / Group / Pooling 25

1 View at a glance

One-line summary of this part: Convolution is an "operation that parameterically encodes 2D information," and modern vision models have emerged by dilating, transposing, separating (grouping), and extending this encoding method to different input/output structures.

Proceed by following the map below.

```
flowchart TD
    A["복습: Conv / Stride / Padding / Pooling"] --> B["Dilated (atrous) Convolution"]
    A --> C["Transposed Convolution<br/>(deconvolution)"]
    C --> C1["Pixel-to-pixel 학습<br/>FCN / Semantic Segmentation"]
    A --> D["효율적 합성곱"]
    D --> D1["Separable Conv"]
    D --> D2["Group Conv"]
    A --> E["Vanilla CNN을 넘어서"]
    E --> E1["구조적 출력<br/>label / pixel / graph"]
    E --> E2["다양한 입력<br/>spectrogram / AlphaGo"]
    E --> E3["Random / Unsupervised filter"]
    A --> F["신경과학적 기반 (V1)"]
    A --> G["ImageNet과 CNN의 부흥<br/>LeNet→AlexNet→GoogLeNet→ResNet"]
    G --> H["What's next: Transformer / ViT / gMLP"]
```

> How to use this note
>
> Each section is structured to be read in the order of Direct Definition → Mathematical Derivation → Geometric/Statistical Intuition → Analogies for Wireless Communication and Signal Processing. It is recommended to copy the formulas by hand once and then verify them by running them directly using the NumPy code in the final Section 11.

---

2 Review: Basic Convolution Operations and Pooling

Before proceeding with full-scale expansion, we will re-examine the four quantities defining a convolutional layer—stride, padding, output size, and number of parameters—and pooling. This is a common area in exams that requires manual calculation.

2.1 Convolution is actually cross-correlation

The operation called "convolution" in deep learning is not the true convolution of signal processing (flipping the kernel), but cross-correlation. The output (i, j) component in a single channel is

$$y[i,j] \;=\; \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} x[\,i\,s+m,\;\; j\,s+n\,]\; w[m,n],$$

is the inner product of the input patch and the filter. Since the kernel is not flipped, the learned w can be viewed as learning an "already flipped kernel," and consequently, there is no difference from a learning perspective.

> **i** The fact that inner product equals similarity (determines the pooling type later)
>
> $$\langle \mathrm{w}, \mathrm{x} \rangle = \|\mathrm{w}\|\,\|\mathrm{x}\| \cos\theta$$
>
> The cross-correlation value between filter w and input patch x measures the directional alignment (angular distance). If the value is $\theta \to 0$, it is maximum $\to$ a strong response indicating "this pattern is at this location." This perspective serves as the criterion for distinguishing between max and min pooling in Section 2.6.

2.2 Stride: Decimation from a multirate perspective

Stride s applies the filter by skipping s pixels. Since adjacent responses are highly correlated (redundant), for high-resolution (4K, 8K) inputs, redundant calculations are reduced and downsampling is performed to $s > 1$.

- e.g.: 7×7 input + 3×3 filter.
  - $s = 2 \to$ Prints $\{0, 2, 4\}$ neatly aligned with one axis at position 3×3.

- It matches only up to $s = 3 \rightarrow \{0, 3\}$, and information is truncated at the boundary (compensated by padding below).

2.3 Padding: Preservation of boundary information

Adding virtual values to the edges so that the filter extends beyond the boundaries. Three types are common.

Method	Fill Value	Effect/Remarks
Zero padding	0	Most common. Effectively "turns off" the weights multiplied at boundaries → No noise injection. (Black border if value range $[0, 255]$)
Replicate(repeat) padding	Duplicate boundary pixels	Extend boundary values as they are $(\dots, 0.5, 0.5 \mid \text{image} \mid 0.5, 0.5, \dots)$
Reflect(mirror) padding	Boundary-based mirror reflection	Symmetric reflection like $\dots, 1, 0.5 \mid \text{image} \mid 0.5, 1, \dots$

Zero padding is based on exactly the same concept as zero-padding in FFT / boundary processing for linear convolution. With appropriate padding, the output size can be matched to the input ("same" padding).

2.4 Output Size Formula

$$\boxed{o = \left\lfloor \frac{i - k + 2p}{s} \right\rfloor + 1}$$

where i input size, k filter size, p one-sided padding, s stride. The library handles this automatically, but manual calculation will appear on the exam.

! Representative Example (Must be done by hand)

Input 32×32, Filter 5×5, Stride 1, Padding 2, Filter 10.

$$o = \frac{32 - 5 + 2 \cdot 2}{1} + 1 = 31 + 1 = 32.$$

→ The output tensor is $32 \times 32 \times 10$ (number of filters = number of output channels).

2.5 Counting parameters (CNN vs MLP)

One filter parameter = (kernel volume) + (1 bias). 5×5 If you apply the filter to a 3-channel input

$$\underbrace{5\times5\times3}_{=75}+\underbrace{1}_{\text{bias}}=76\quad(\text{필터당}),\qquad 76\times10\text{ filters}=\boxed{760}.$$

> **Common pitfall: The filter depth automatically matches the input channels**
>
> When saying "'5×5 filter'," usually only the spatial size is mentioned. If the input is 3 channels, the filter automatically becomes $5\times5\times3$. The bias is one scalar per filter ($W*x+b$'s b). Therefore, it is 760, not 250.

If you process and $32\times32\times3$ inputs using MLP, each output neuron is connected to every pixel

$$32\times32\times3=3072\ (+1\text{ bias}=3073)\quad(\text{뉴런 1개당})$$

weights are required. As the number of output neurons increases, it explodes through multiplication. On the other hand, convolution shares all locations with 10 filters and creates $32\times32\times10$ feature maps even after training only 760.

The nature of this gap is two inductive biases.

1. Weight Sharing: Reuse the same filter at all locations → Automatic encoding of location-invariant patterns, drastic reduction in parameters.
2. Local connectivity: Views only at the patch level → No need to "learn inter-pixel relationships individually," simultaneously securing data efficiency + computational efficiency.

During training, backpropagation updates the filter not at a single location, but by summing the gradients of all locations where the filter has slid (a direct consequence of weight sharing):

$$\frac{\partial L}{\partial w[m,n]}=\sum_{i,j}\frac{\partial L}{\partial y[i,j]}\,x[\,is+m,\ js+n\,].$$

> **Wireless Communication & Signal Processing Analogy — Matched Filters and LTI Systems**
>
> A convolutional layer is a linear time-invariant (LTI) filter bank for an input signal. Each filter is a matched filter for a specific pattern, and the larger the correlation (inner product) with the patch, the larger the output — this is the same principle as correlation with known templates (pilot/preamble) at the receiver maximizing the SNR. Weight sharing is the time-invariance itself, meaning "tap coefficients do not change over time."

2.6 Pooling: An operation to select a number from a pool

"Pooling" is an operation that selects a single number from a pool of numbers (English noun pool → verb pooling). It selects a representative value from a pool of responses obtained by applying filters to multiple locations.

Type	Operation	When is it suitable
Max pooling	$\max_k x_k$	When the filter response is similarity (inner product). Keep only the best fit
Average(mean) pooling	$\frac{1}{n}\sum_k x_k$	When you want to preserve background/global statistics. Low-pass (smoothing)
Min pooling	$\min_k x_k$	When the filter response is distance (dissimilarity), the closest (smallest) response is the best match

> ⚠ Terminology Note: mean vs min
>
> mean = average, min = minimum. The pronunciation is confusing, but they are completely different. average pooling = mean pooling, minimum pooling = min pooling.

Why the types differ (statistical and geometric basis). As seen in Section 2.1, cross-correlation measures similarity $\langle \mathrm{w}, \mathrm{x} \rangle = \|\mathrm{w}\|\|\mathrm{x}\| \cos\theta$. Therefore, the "strongest pattern match" is the max value → max pooling. However, if the filter produces dissimilarity such as Euclidean distance $\|\mathrm{w} - \mathrm{x}\|_2$, the best match is the minimum value → min pooling. The nature of the operation determines the pooling.

Statistically speaking, average pooling estimates the expected value (low variance, bias toward the background) of the response, while max pooling estimates the extreme value (high variance, detection of prominent activity), which is a kind of bias-variance trade-off.

Three benefits and one loss of max pooling.

1. Expanding the receptive field — without increasing parameters. Stacking conv-pools allows the same 3×3 filter to see a wider area in the input coordinate system (Section 2.6.1).
2. Removal of redundant/noise responses. Since responses at adjacent locations are strongly correlated (almost redundant), only the maximum value is passed to the next layer → suppression of unnecessary noise, saving computation.
3. Local translation invariance.
4. Disadvantage: Some accurate location information is lost. This is acceptable for classification, but for tasks where location is important (detection/segmentation), average pooling or pooling omission is chosen.

Because of this balance of "positive effects ≫ negative effects," pooling is widely used not only in CNNs but also in MLP and Transformer families.

2.6.1 Recurrence relation

The reception area of ZZZPH0007803ZZZ layer l r_lZ and "jump" J_l is

$$r_l = r_{l-1} + (k_l - 1)\, J_{l-1}, \qquad J_l = J_{l-1}\, s_l, \qquad r_0 = 1,\ J_0 = 1.$$

Configuration	r (Area of Reception)
conv3(s1)	3
conv3(s1) → conv3(s1)	5
conv3(s1) → pool2(s2) → conv3(s1)	8
conv3 → pool2 → conv3 → pool2 → conv3	18

i What "pooling gives a wider view" really means

Intuitively, it is said that if 3×3 conv is followed by 2×2 pooling, the next conv will look at the 6×6 degree of the input, but the recurrence relation provides the exact value considering overlap (8 in the table above). The key point is that pooling multiplicatively increases the jump J to rapidly expand the receptive range. By expanding the receptive range with pooling instead of using large filters, a wide context is obtained without parameter or data cost.

Wireless Communication & Signal Processing Analogy — Decimation and Envelope Detection in Multirate Systems

Pooling corresponds to decimation in multirate DSPs. Redundancy is eliminated by lowering the sample rate through stride/pooling. In particular, max pooling is equivalent to envelope detection after rectification, and average pooling is equivalent to low-pass averaging. "Large filter = long tap for narrow bandwidth selectivity," and by lowering the resolution through pooling and then reusing the same short filter, wider frequency and spatial selectivity are obtained with the same number of taps.

3 Dilated (Atrous) Convolution

One-line definition: A convolution that increases the range of receptacle without increasing parameters by placing holes between filter tabs. atrous comes from the French à trous ("having holes").

3.1 Definition and effective kernel size

For dilation rate d, place filter tabs at d cell intervals. $k \times k$ The effective size of the filter is

$$k_{\text{eff}} = k + (k-1)(d-1) = d(k-1) + 1.$$

Example: $k = 3,\ d = 2 \Rightarrow k_{\text{eff}} = 2 \cdot 2 + 1 = 5$. That is, the parameters are still $3 \times 3 = 9$, but they cover the 5×5 region in the input coordinate system. The output size is

$$o = \left\lfloor \frac{i - k_{\text{eff}} + 2p}{s} \right\rfloor + 1.$$

3.2 What is the difference from Stride?

```
flowchart LR
    subgraph Stride["Stride=2 (계산을 건너뜀)"]
      direction LR
      s["픽셀 일부는 합성곱 계산에서 제외됨"]
    end
    subgraph Dilated["Dilated rate=2 (계산은 모두 보되 성기게)"]
      direction LR
      di["모든 픽셀이 어느 출력엔가 기여, 단 탭은 성기게 샘플링"]
    end
```

- Stride: Decimates output positions → Some input pixels may be discarded as they do not contribute to any output.
- Dilated: Maintain output resolution but sparse tabs → Expand the receptacle area without discarding entire pixels.

Therefore, dilated is suitable when you "want to skip (view a wide area) but do not want to remove any entire pixels." It is particularly useful in high resolutions, such as 4K and satellite imagery, where adjacent pixels contain nearly identical geometric information (choose between large max pooling or dilated).

💡 Wireless Communication & Signal Processing Analogy — Sparse Tap FIR / Comb Filter

A dilated conv is a zero-stuffed / sparse-tap filter with zeros inserted between the taps. It has a structure similar to a filter upsampled in polyphase decomposition or a comb filter with a periodic response, and increases spatial selectivity by expanding the support of the impulse response without increasing the number of taps (=parameters).

---

4 Transposed Convolution (Deconvolution)

One-line definition: The operation itself is identical to convolution, but it is a convolution used in the direction of "upsampling" the compressed representation back to its original size. The name deconvolution is misleading, and the correct name is transposed convolution.

4.1 Why "transposed" — Transposition of Linear Transformations

Since convolution is a linear operation, for the input vector x, to the sparse (Toeplitz) matrix C

$$\mathrm{y} = C\,\mathrm{x}, \qquad C \in \mathbb{R}^{(\text{작은 출력})\times(\text{큰 입력})}$$

It can be written as . Transposed convolution applies $C^\top$ created with the same weights

$$\mathrm{z}' = C^\top \mathrm{z}, \qquad C^\top \in \mathbb{R}^{(\text{큰 입력})\times(\text{작은 출력})}$$

, that is, it returns from small (output) space → large (input) space. The name is natural given that the gradient of the forward convolution is exactly the product of $C^\top$. If there is no information loss (e.g., max pooling), the original space can be (theoretically) restored using appropriate inverse weights.

> **i** Mathematical Reinforcement (Linear Algebra)
>
> If the matrix of a linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ is C, then the matrix of its adjoint T^* is $C^\top$ ($\langle C\mathrm{x}, \mathrm{z}\rangle = \langle \mathrm{x}, C^\top \mathrm{z}\rangle$). A transposed conv is the application of the adjoint of a convolution operator, which corresponds exactly to the analysis–synthesis pair in signal processing.

4.2 Transposed conv with stride and its uses

Downsampled by the stride in the forward direction, zeros are inserted between the inputs in the transposed direction, and then convolution is performed to upsample. A typical use case is pixel-to-pixel learning.

4.2.1 Fully Convolutional Network (FCN) and Semantic Partitioning

When the goal is to produce a “semantic map of the same size as the input, where each pixel has a class label”:

```
flowchart LR
    I["입력 H×W×3"] --> E["인코더<br/>conv+pool (다운샘플)"]
    E --> Z["병목: 작고 의미적 · 잡음제거된 표현"]
    Z --> D["디코더<br/>transposed conv (업샘플)"]
    D --> O["출력 H×W×C<br/>픽셀별 라벨"]
```

- You can stack convolutions of the same size all the way to the end, but this fails to compress information in the middle, resulting in carrying noise all the way to the end and causing the side effect of having to enlarge the filter to see a wide context.
- Instead of , if you compress with an encoder (reducing dimensions + removing noise through pooling) → restore to the original resolution via transposed convolution with a decoder, intermediate parameters are reduced and data efficiency improves. In this case, each pixel location no longer has RGB values but semantic values.

> 💡 Wireless Communication & Signal Processing Analogies — Intermediate Filter / Synthetic Filter Bank / DAC Reconstruction
>
> Transposed convolution is the upsampling (zero insertion) + interpolation filter itself. It corresponds to the synthesis side of an analysis-synthesis filter bank or the reconstruction filter of a DAC. It is a structure where the encoder (analysis) decomposes and compresses into subbands, and the decoder (synthesis) combines them again to restore the original signal dimension.

---

5 Beyond Vanilla CNN 1: Efficient Convolution

5.1 Separable Convolution

One-line definition: A technique to reduce the number of multiplications by decomposing a 2D convolution into cascade operations of two 1D convolutions.

If the kernel is separable (rank-1), i.e., $K = \mathrm{u}\,\mathrm{v}^\top$, then

$$K * x = \mathrm{u} *_{\mathrm{row}} (\mathrm{v} *_{\mathrm{col}} x).$$

The number of multiplications per output position decreases from $k \times k$ to $2k$ (if it is $k = 3$, it becomes $9 \to 6$).

> ⚠ Why isn't it widely used? — Parallelization cost
>
> Every efficiency improvement comes at a cost. Separable operations are dependent operations (first a 1D convolution → then another 1D convolution on the output), so even with 9 cores, they cannot all be used simultaneously and must wait for the output of the first stage. In other words, although the amount of computation (FLOPs) decreases, parallelization efficiency is poor, resulting in a loss in wall-clock time. In the GPU era, parallelism equals speed, so it is not widely adopted.

> ℹ Term distinction (accuracy)
>
> Here, "separable" refers to spatially separable: decomposing $k \times k$ into $k \times 1$ and $1 \times k$. This is a different concept from depthwise separable, which is common in MobileNet and the like — channel-wise spatial conv + 1×1 pointwise conv —. The latter is much more widely used in practice.

5.2 Group Convolution

One-line definition: A technique that divides input channels into g groups, concatenates each group independently, and then concatenates them. Advantageous for parallelization.

If the C_{in} channels are split into g groups, each filter only examines the C_{in}/g channel, so the kernel term parameter FLOPs are reduced to approximately $1/g$. Since the groups are independent of each other, they can be parallelly deployed across multiple cores/GPUs (the original example used by AlexNet for 2-GPU splitting).

$$\text{full: } (k^2 C_{\text{in}} + 1) C_{\text{out}} \xrightarrow{g \text{ groups}} \left(k^2 \frac{C_{\text{in}}}{g} + 1\right) C_{\text{out}}.$$

e.g.: $k = 3,\ C_{\text{in}} = C_{\text{out}} = 64,\ g = 4 \Rightarrow 36{,}928 \to 9{,}280$ (approx. $\times \frac{1}{4}$).

⚠ Cost — Inter-group interaction loss

Cross-channel interaction is interrupted by the extent that the groups are divided. When combining group outputs, some information is lost with simple averaging/connection, so unlike separable, the operations are not equivalent. To mitigate this, channel shuffling (ShuffleNet) or interleaved group conv (Zhang et al., ICCV 2017) have been proposed.

	Separable conv	Group conv
Decomposition Axis	Spatial Axis ($k \times 1$, $1 \times k$)	Channel Axis (Group Split)
Operational Equivalence	Equivalent (no information loss)	Non-equivalent (loss of inter-group interaction)
Parallelization	Bad (Dependent Operations)	Good (Group Independence)
Main Gain	FLOPs Decrease	Parallelization + Parameter Reduction

💡 Wireless Communication & Signal Processing Analogy — Separated 2D Filter vs. Subband/Antenna-based Processing

Separable convolution is isomorphic to the separable 2D filter (row-column decomposition, separable FFT) in image processing. Group convolution corresponds to creating parallel branches in a block-diagonal structure, similar to parallel processing per antenna/stream in MIMO or per-subcarrier processing in OFDM (parallel gain at the cost of breaking inter-branch coupling).

6 Beyond Vanilla CNN 2: Structured Input/Output and Input Forms

6.1 Structured outputs

The structure of the CNN output varies depending on the task.

Output Format	Assignment	Example
Single Label	Classification	Image Classification
Labels by pixel	Semantic segmentation	FCN (Section 4.2.1)
Graph (Matrix)	Graph Prediction	Structure represented by an adjacency matrix

The graph output is represented in 2D as an adjacency matrix. However, since the rows and columns of the adjacency matrix contain the semantic relationships of nodes, the structural information is broken if handled with a standard 3×3 conv → Filter and output design must be done differently, like a Graph Neural Network (GCN/GNN).

6.2 Various input types (Input data types)

CNN can receive "anything arranged in 2D".

- Spectrogram: Processing the time-frequency representation (STFT) of a voice or communication signal as a 2D image.
- AlphaGo Input: 19×19 A 17 tensor stacking $19 \times 19 \times 17$ feature planes on a Go grid. The planes encode the current and past stone arrangements and turn colors:

$$[X_t, Y_t, X_{t-1}, Y_{t-1}, \ldots, X_{t-7}, Y_{t-7}, C] \quad (8+8+1=17).$$

  This tensor is encoded into a CNN like a single "image" and then combined with MCTS and reinforcement learning to estimate the winning probability of the next move.

> Wireless Communication & Signal Processing Analogy — Time-Frequency Image
>
> Spectrograms are a core tool for wireless signal analysis. In modulation recognition, interference detection, and radar processing, viewing STFT/scalograms as 2D images and applying CNNs is essentially the task of detecting local patterns with matched filters in the time-frequency plane.

7 Beyond Vanilla CNN 3: Random / Unsupervised Filter

So far, we have learned filters through supervised learning, but there is another way.

- Random Filters: Filters are ultimately basis that encode the input. If a sufficient number of random filters are selected, they may include a subset that is nearly orthogonal (uncorrelated) to one another, which can exhibit expressive power comparable to that of a trained filter (Jarrett 2009; Saxe 2011; Pinto 2011; Cox & Pinto 2011). Instead of skipping the training process, more filters (→ increased inference operations) are usually required for equivalent performance.
- Unsupervised Learning Filters: Learn filters without labels. A representative example is contrastive learning —
  - Patch extracted from the same image = positive,
  - Patch extracted from another image = Voice (negative) — Learn the expression by setting it to (extend to CLIP/SigLIP, etc. later).

> 💡 Wireless Communication & Signal Processing Analogy — Random Projection / Compressed Sensing
>
> The success of random filters is the same intuition as the random measurement matrix (satisfying RIP) of compressed sensing. A well-designed random basis preserves the information of a signal with almost no loss — this is isomorphic to how random codes or random projections become efficient encoding means.

---

8 The neuroscience basis of CNN

The computation of CNN is similar to the processing of the primary visual cortex (V1, primary visual cortex).

- V1 has a 2D structure for processing visual signals.
- Simple cells ≈ Linear operations (multiplication and sum) of CNN.
- complex cells ≈ pooling unit of CNN (non-linear, local maximum).
- The first layer filter learned by CNN through backpropagation converges to a shape similar to the Gabor filter (directional, local frequency selectivity, not isotropic) known as the V1 mechanism.

However, the human visual system and CNN are very different.

	Visual brain	CNN
Resolution	Low (excluding fovea center)	Relatively High
Sensory Integration	Integration with Other Senses	Pure Vision Only
Dimensions	Rich 3D Information	2D Only
Operation mechanism	Unknown	Multiplication·Addition

(Evidence of 3D perception: If you play table tennis with one eye covered, you cannot gauge the depth or timing of the ball — because 3D is reconstructed using binocular parallax.) See also Goodfellow et al., Deep Learning §9.10.

> 💡 Wireless Communication & Signal Processing Analogy — Gabor Atom and Time-Frequency Localization
>
> A Gabor filter is like a Gabor atom (Gaussian window × sinusoidal wave) of time-frequency analysis. It acts as a matched filter for local chirp/tone detection, serving as a basis for optimally compromising time-frequency localization in the uncertainty limit.

9 Why CNN Rises: ImageNet and a Brief History

9.1 ImageNet — Large image dataset

- Built by Fei-Fei Li (currently at Stanford) in 2009 while at Princeton. Inspired by WordNet (Princeton's word ontology tree), it collected images corresponding to each word node.
- Scale: Approx. 14M images, 24k categories. Collected via search and noise refined via crowdsourcing (e.g., "apple" contains fruit/company).
- Challenge subset (ImageNet-1K): Approx. 1.2M images, 1000 categories. The competition was held, explosively boosting vision research.

> ImageNet Challenge Stagnation (2009~2011)
>
> The error rate of the initial best algorithm appeared to stagnate, improving slowly by 1-3 percentage points annually from $\sim 35\%$ (accuracy $\sim 64\%$). There was even skepticism that this was due to label noise — soon CNNs were on the verge of turning the tables.

9.2 LeNet → AlexNet → Deeper

```
timeline
    title CNN 아키텍처의 흐름
    1998 : LeNet (Yann LeCun) - 손글씨 숫자(MNIST)·우편번호 인식
    2012 : AlexNet (Krizhevsky, Hinton) - ImageNet 오류율 급감, 부흥의 시작
    2014 : GoogLeNet (Inception) - 매우 깊은 망
    2015 : ResNet (Kaiming He) - 잔차연결로 초심층 학습
    2017 : DenseNet - 조밀 연결
    2021~ : ViT / gMLP - Transformer·MLP 계열로 전환
```

- LeNet (1998): Proposed by LeCun for handwritten digit recognition. 5 convolutional layers + a final MLP structure. Classified digits using MNIST and utilized for zip code recognition. (The name comes from Le in LeCun.)
- AlexNet (2012): Krizhevsky, under the supervision of Hinton (Toronto), constructed a 7-layer structure (5 convolutional layers + 2 fully connected layers) similar to LeNet, and dropped the error rate by about 10%p in just one year on the ImageNet challenge (a drop that would have taken more than 5 years under existing trends). A signal of revival.

- GoogLeNet (2014, Inception): An Inception module combining filters of various sizes in parallel. Trained for over a week on multiple high-performance GPUs. (Typically counted as a depth of 22 layers.)
- ResNet (2015, Microsoft Research Asia, Kaiming He): Stably trained up to 152 layers (experimentally over 1000 layers) using skip connections. Rated as "ultra deep" by LeCun.
- DenseNet (2017): Densely connects all front layers to back layers.

> **i** Why Residual Connections Enable Deep Networks (Causal Chains)
>
> The Jacobian of the residual block $y = x + f(x)$ is $\frac{\partial y}{\partial x} = I + \frac{\partial f}{\partial x}$. Due to the identity term I, gradients are preserved during backpropagation rather than being multiplied and vanished → very deep networks can be learned → improved expressiveness and accuracy. (In other words, a chain of identity Jacobian term → gradient preservation → learnability.)

The software ecosystem also exploded: DeCaf → Caffe(Yangqing Jia) → TensorFlow(Google) / Torch(LeCun) → PyTorch(current PyTorch Foundation).

9.3 What's next? Transformer / ViT / gMLP

- Transformer (Vaswani et al., NeurIPS 2017): The standard architecture of modern NLP. Attention Is All You Need.
- Vision Transformer (ViT) (Dosovitskiy et al., ICLR 2021): Views the image as a 16×16 patch token sequence and processes it with a Transformer. "An image is worth 16×16 words."
- gMLP (Liu et al., NeurIPS 2021): Pay Attention to MLPs — Competitiveness without attention with gated MLPs. "Attention may not be everything."

ViT is now largely replacing CNN in many 2D encoding tasks.

---

10 Key Summary

- CNN is a method that efficiently encodes 2D information into parameters and data, and its secret lies in the inductive bias of weight sharing + local connections.
- Components: conv layer + pooling layer + MLP. Output size $o = \lfloor (i-k+2p)/s \rfloor + 1$, parameters per filter $= k^2 C_{\text{in}} + 1$.
- Dilated: Expanded receptacle without parameters (sparse tab). Transposed: Upsampled by the transpose of the convolution ($C^\top$) (FCN/splitting).
- Efficiency: separable(spatial decomposition, weak equivalence/parallelization) / group(channel decomposition, strong non-equivalence/parallelization).
- CNN is extended to structured inputs and outputs (graphs → GNNs) and various inputs (spectrosograms, AlphaGo), and can also be trained with random/unsupervised filters, and resembles V1.
- ImageNet + AlexNet sparked the revival → GoogLeNet/ResNet/DenseNet → Now the weight has shifted to ViT/gMLP.

Operation	Parameter change	Capacity/Resolution	Key keyword
Stride ↑	Invariant	Resolution ↓	Decimation, Deduplication
Pooling	0 (No Training)	Pooling ↑, Resolution ↓	Invariance, Noise Reduction
Dilated	Invariant	Range ↑ (Resolution maintained)	à trous, Sparse tab
Transposed	(by filter)	Resolution ↑	Transpose/Adjunctive, Upsampling
Separable	↓ (FLOPs)	Invariant	Spatial decomposition, Dependency
Group	↓ $(1/g)$	Invariant	Channel decomposition, Parallel

More to read

- Dive into Deep Learning (d2l.ai): Convolutional Basics (Padding, Stride, Multichannel, Pooling), Modern CNNs (AlexNet, VGG, NiN, GoogLeNet, BatchNorm, ResNet, DenseNet), Transpositional Convolution and Semantic Partitioning (FCN) in the Computer Vision Field.
- Goodfellow, Bengio, Courville, Deep Learning, Ch. 9 (especially §9.10 Neuroscience Basis).

- G. Thomas, Mathematics for Machine Learning: Inner Product, Norm, Orthogonality, Linear Transformations and Transpositions (Adjointities) — The Mathematical Foundations of Similarity/Distance in Section 2.6 and Transposed Convolution in Section 4.1.

---

11 Appendix: Check directly with NumPy

The code below runs as is without dependencies (only `numpy`). It is intended for manual verification of the concept.

11.1 Output size·Parameters·Area

```
import numpy as np

def out_size(i, k, s=1, p=0, d=1):
    """conv 출력 크기. d>1이면 dilated."""
    k_eff = d * (k - 1) + 1
    return (i - k_eff + 2 * p) // s + 1

def conv_params(kh, kw, c_in, c_out):
    """필터당 (kh*kw*c_in + 1), 총 *c_out."""
    return (kh * kw * c_in + 1) * c_out

def receptive_field(layers):
    """layers: [(k, s), ...]  반환: 입력 좌표계 수용영역."""
    r, J = 1, 1
    for (k, s) in layers:
        r = r + (k - 1) * J
        J = J * s
    return r

print(out_size(32, 5, s=1, p=2))                # 32
print(out_size(7, 3, s=2, p=0))                 # 3
print(out_size(7, 3, s=1, p=0, d=2))            # 3 (k_eff=5)
print(conv_params(5, 5, 3, 10))                 # 760
print(32 * 32 * 3 + 1)                          # 3073  (MLP 뉴런 1개)
print(receptive_field([(3,1),(2,2),(3,1)]))     # 8
```

11.2 2D cross-correlation using im2col method (including dilated)

```
def conv2d(x, w, stride=1, pad=0, dilation=1):
    H, W = x.shape
    kh, kw = w.shape
    kh_e = dilation * (kh - 1) + 1
    kw_e = dilation * (kw - 1) + 1
    xp = np.pad(x, pad, mode='constant')
    Hp, Wp = xp.shape
    oh = (Hp - kh_e) // stride + 1
    ow = (Wp - kw_e) // stride + 1
    y = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            r0, c0 = i * stride, j * stride
            patch = xp[r0:r0+kh_e:dilation, c0:c0+kw_e:dilation]
            y[i, j] = np.sum(patch * w)        # 패치와 필터의 내적
    return y

x = np.arange(1, 26).reshape(5, 5).astype(float)
w = np.ones((3, 3))
print(conv2d(x, w).shape)                  # (3, 3)
print(conv2d(x, w, dilation=2).shape)      # (1, 1)
print(conv2d(x, w, stride=2, pad=1).shape) # (3, 3)
```

11.3 Transposed convolution = Transpose of the convolution matrix

```
# 1D 예시: 순방향 conv를 행렬 C로 쓰고, 전치 C^T로 업샘플
x_in = np.array([1., 2., 3., 4.])
ker  = np.array([1., 0., -1.])
n_out = len(x_in) - len(ker) + 1
C = np.zeros((n_out, len(x_in)))
for i in range(n_out):
    C[i, i:i+len(ker)] = ker

y  = C @ x_in            # 순방향(다운): 4차원 -> 2차원
up = C.T @ np.array([1., 1.])   # 전치(업): 2차원 -> 4차원
print(y)                 # [-2. -2.]
print(up)                # [ 1.  1. -1. -1.]
```

11.4 Separable / Group / Pooling

```
# spatially separable: K = u v^T 이면 2D conv = (1D row) o (1D col)
u = np.array([1., 2., 1.]); v = np.array([1., 0., -1.])
```

```
K = np.outer(u, v)
full = conv2d(x, K)
sep  = conv2d(conv2d(x, u.reshape(3, 1)), v.reshape(1, 3))
print(np.max(np.abs(full - sep)))          # ~0.0 (동치)

# group conv 파라미터: g배 절감
def params(kh, kw, c_in, c_out, g=1):
    return (kh * kw * (c_in // g) + 1) * c_out
print(params(3, 3, 64, 64, g=1))           # 36928
print(params(3, 3, 64, 64, g=4))           # 9280

# max / average pooling
def pool(x, k=2, mode='max'):
    H, W = x.shape
    oh, ow = H // k, W // k
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            blk = x[i*k:(i+1)*k, j*k:(j+1)*k]
            out[i, j] = blk.max() if mode == 'max' else blk.mean()
    return out

xp = np.arange(1, 17).reshape(4, 4).astype(float)
print(pool(xp, 2, 'max'))   # [[ 6.  8.] [14. 16.]]
print(pool(xp, 2, 'avg'))   # [[ 3.5  5.5] [11.5 13.5]]
```